

vLLM专题（二）：安装-CPU



大模型专题系列 专栏收录该内容



您已是超级会员，正在免费阅读会员专享内容

vLLM CPU支持是可行但需要从源码构建
了解并合理配置环境变量对性能至关重要
针对CPU架构特点进行优化（如解耦、并行策略）
是关键。

虽然不如GPU原生支持，但精心配置，vLLM在
CPU上也能提供强大的大模型服务能力

¥49.90
¥99.00

111 篇文章

已订阅

8折续费

vLLM 是一个 Python 库，支持以下 CPU 变体。选择您的 CPU 类型以查看供应商特定的说明：

Intel/AMD x86

vLLM 最初支持在 x86 CPU 平台上进行基本模型推理和服务，支持的数据类型包括 FP32、FP16 和 BF16。

注意

此设备没有预构建的 wheel 包或镜像，因此您必须从源代码构建 vLLM。

1. 要求

Python 版本 3.9 至 3.12

Intel/AMD x86

操作系统: Linux

编译器: gcc/g++ >= 12.3.0（可选，推荐）

指令集架构 (ISA): AVX512（可选，推荐）

像是CPU的特技，能
加速特定类型的计算
，尤其针对BF16这种
混合精度格式，在大
模型推理中常用

提示

Intel Extension for PyTorch (IPEX) 为 PyTorch 扩展了最新的功能优化，以在 Intel 硬件上提供额外的性能提升。

2. 使用 Python 设置

2.1 创建一个新的 Python 环境

您可以使用 conda 创建一个新的 Python 环境：

```
# (Recommended) Create a new conda environment.  
conda create -n myenv python=3.12 -y  
conda activate myenv
```

注意

PyTorch 已废弃 conda 发布渠道。如果您使用 conda，请仅用于创建 Python 环境，而不是安装软件包。

或者，您可以使用 uv 创建一个新的 Python 环境，uv 是一个非常快速的 Python 环境管理器。请按照文档安装 uv。安装 uv 后，您可以使用以下命令创建一个新的 Python 环境：

```
# (Recommended) Create a new uv environment. Use `--seed` to install `pip` and `setuptools` in the environment.
uv venv myenv --python 3.12 --seed
source myenv/bin/activate
```

3. 预编译的 wheel 包

目前，没有预构建的 CPU wheel 包。

3.1 从源代码构建 wheel 包

Intel/AMD x86

首先，安装推荐的编译器。我们建议使用 gcc/g++ >= 12.3.0 作为默认编译器，以避免潜在的问题。例如，在 Ubuntu 22.4 上，您可以运行：

```
sudo apt-get update -y
sudo apt-get install -y gcc-12 g++-12 libnuma-dev
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-12 10 --slave /usr/bin/g++ g++ /usr/bin/g++-12
```

把新装的编译器设置为默认的

其次，安装用于 vLLM CPU 后端构建的 Python 包：

指定Pytorch的CPU版本仓库

```
pip install --upgrade pip
pip install "cmake>=3.26" wheel packaging ninja "setuptools-scm>=8" numpy
pip install -v -r requirements-cpu.txt --extra-index-url https://download.pytorch.org/whl/cpu
```

专门为CPU后端准备的依赖列表

最后，构建并安装 vLLM CPU 后端：

```
VLLM_TARGET_DEVICE=cpu python setup.py install
```

注意

AVX512_BF16 是一个扩展的指令集架构（ISA），提供原生的 BF16 数据类型转换和向量乘积指令，相较于纯 AVX512，能带来一定的性能提升。CPU 后端构建脚本会检查主机的 CPU 标志，以确定是否启用 AVX512_BF16。

如果您希望强制启用 AVX512_BF16 进行交叉编译，请在构建之前设置环境变量

VLLM_CPU_AVX512BF16=1。

4. 使用 Docker 设置

4.1 预构建的镜像

目前，没有预构建的 CPU 镜像。

4.2 从源代码构建镜像

```
docker build -f Dockerfile.cpu -t vllm-cpu-env --shm-size=4g .
docker run -it \
```

提示

对于 ARM 或 Apple Silicon，请使用 Dockerfile.arm。

5. 支持的功能

vLLM CPU 后端支持以下 vLLM 功能：

- 张量并行 (Tensor Parallel)
- 模型量化 (INT8 W8A8、AWQ、GPTQ)
- 分块预填充 (Chunked-prefill)
- 前缀缓存 (Prefix-caching)
- FP8-E5M2 键值缓存 (KV-Caching) (待实现)

优化技术，能让模型在多个设备或节点上并行计算

目的是加速生成过程中注意力计算，用更低的精度格式存储键值对，节省内存越大，理论上处理请求越多。

压缩模型精度（权重），大幅减少内存占用和计算量

vllm的核心优势，提升处理请求文本的吞吐量，更聪明处理输入序列，减少重复计算。

6. 相关的运行时环境变量

VLLM_CPU_KVCACHE_SPACE：指定 KV 缓存的大小（例如，VLLM_CPU_KVCACHE_SPACE=40 表示为 KV 缓存分配 40 GB 空间），设置更大的值将允许 vLLM 同时处理更多请求。此参数应根据用户的硬件配置和内存管理模式进行设置。

VLLM_CPU_OMP_THREADS_BIND：指定用于 OpenMP 线程的 CPU 核心。例如，VLLM_CPU_OMP_THREADS_BIND=0-31 表示会将 32 个 OpenMP 线程绑定到 0-31 号 CPU 核心。VLLM_CPU_OMP_THREADS_BIND=0-31|32-63 表示会有 2 个张量并行进程，rank0 的 32 个 OpenMP 线程绑定到 0-31 号 CPU 核心，rank1 的 OpenMP 线程绑定到 32-63 号 CPU 核心。

7. 性能优化建议

我们强烈推荐使用 TCMalloc 进行高性能内存分配和更好的缓存局部性。例如，在 Ubuntu 22.4 上，您可以运行：

默认的malloc在高并发、大数据下可能效率不高
TCMalloc当作高效内存管家

关系CPU核心利用效率，OpenMP并行编程接口，用来并行计算。这个变量告诉OpenMP哪些并行线程绑定到那些CPU核心上跑，好处是减少线程在不同核心间跳来跳去带来的开销，提高缓存命中率。

```
sudo apt-get install libtcmalloc-minimal4 # install TCMalloc Library
find / -name *libtcmalloc* # find the dynamic link library path
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libtcmalloc_minimal.so.4:$LD_PRELOAD # preload the library to LD_PRELOAD
python examples/offline_inference/basic.py # run vLLM
```

在使用在线服务时，建议为服务框架保留 1-2 个 CPU 核心，以避免 CPU 超负荷。例如，在具有 32 个物理 CPU 核心的平台上，为框架保留 CPU 30 和 31，并将 CPU 0-29 用于 OpenMP：

```
export VLLM_CPU_KVCACHE_SPACE=40
export VLLM_CPU_OMP_THREADS_BIND=0-29
vllm serve facebook/opt-125m
```

如果在具有超线程技术的机器上使用 vLLM CPU 后端，建议使用 VLLM_CPU_OMP_THREADS_BIND 将每个物理 CPU 核心绑定一个 OpenMP 线程。在启用了超线程技术的平台上，具有 16 个逻辑 CPU 核心 / 8 个物理 CPU 核心：

```
lscpu -e # check the mapping between logical CPU cores and physical CPU cores

export VLLM_CPU_OMP_THREADS_BIND=0-7
python examples/offline_inference/basic.py
```

避免过多的线程竞争
同一物理核心的资源

如果在具有 NUMA 架构的多插槽机器上使用 vLLM CPU 后端，请注意使用 VLLM_CPU_OMP_THREADS_BIND 设置 CPU 核心，以避免跨 NUMA 节点的内存访问。

lscpu -e
查看逻辑核心和物理核心对应关系

NUMA 架构下，每个 CPU 核心访问本地内存很快，但访问其它的节点慢很多。

CPU 后端与 GPU 后端有显著不同，因为 vLLM 架构最初是为 GPU 使用优化的。为了提高性能，需要进行一些优化。

将 HTTP 服务组件与推理组件解耦。在 GPU 后端配置中，HTTP 服务和分词任务在 CPU 上运行，而推理在 GPU 上运行，这通常不会造成问题。然而，在基于 CPU 的设置中，HTTP 服务和分词可能会导致显著的上下文切换和缓存效率降低。因此，强烈建议将这两个组件分开，以提高性能。

在启用了 NUMA 的基于 CPU 的设置中，内存访问性能可能会受到拓扑结构的较大影响。对于 NUMA 架构，建议进行两项优化：张量并行（Tensor Parallel）或数据并行（Data Parallel）。

对于具有延迟限制的部署，使用张量并行：根据 GPU 后端设计，Megatron-LM 的并行算法将用于根据 NUMA 节点的数量分割模型（例如，对于一个有两个 NUMA 节点的系统，TP = 2）。通过在 CPU 上合并 TP 特性，张量并行被支持用于服务和离线推理。通常，每个 NUMA 节点被视为一个 GPU 卡。以下是启用张量并行 = 2 的服务示例脚本：

```
VLLM_CPU_KVCACHE_SPACE=40 VLLM_CPU_OMP_THREADS_BIND="0-31|32-63" vllm serve meta-llama/Llama-2-7b-chat-hf -tp=2 --distributed-executor-backend mp
```

使用数据并行（Data Parallel）以获得最大吞吐量：在每个 NUMA 节点上启动一个 LLM 服务端点，并添加一个负载均衡器，将请求分发到这些端点。推荐使用 Nginx 或 HAProxy 等常见解决方案。

Anyscale Ray 项目提供了 LLM 服务的相关功能。以下是使用 Ray Serve 设置可扩展 LLM 服务的示例。